



CENTRO UNIVERSITÁRIO DO PARÁ - CESUPA
ESCOLA DE NEGÓCIOS, TECNOLOGIA E INOVAÇÃO - ARGO

PROCESSAMENTO DE IMAGENS EM LOTE COM PROCESSOS EM PYTHON

**Alberto Eduardo Martins Acosta, Carlos Eduardo Cardoso Silva, Gabriel Costa de
Miranda, Yago Patrick Schnorr Pinto**

BELÉM
2026

1. Problema e dados

Antes de treinar um modelo de visão computacional, as fotos de um dataset precisam ser padronizadas, e o dataset inteiro precisa de estatísticas para normalizar a entrada da rede: a média e o desvio-padrão de cada canal de cor. Este trabalho implementa esse pré-processamento em lote e o paraleliza.

Conjunto de validação do COCO 2017: 5.000 fotografias reais em JPEG, cerca de 800 MB já descompactados. Usamos as 3.334 primeiras em ordem alfabética do nome, número calibrado pelo programa `calibrar.py` para a versão sequencial levar cerca de 4 minutos na máquina da demonstração. A ordem alfabética é fixa, então `--limite 3334` seleciona sempre exatamente as mesmas imagens, em qualquer execução e em qualquer máquina.

Unidade de trabalho: uma imagem. A função `processar_imagem()`, em `pipeline.py`, é chamada pelas duas versões do programa e executa nove etapas para cada foto: decodifica o JPEG, recorta e redimensiona para 512x512 pixels com o filtro LANCZOS, remove ruído com um filtro de mediana 3x3, normaliza o contraste cortando 1% em cada extremo, aplica nitidez (unsharp mask), calcula o histograma RGB com 768 contagens inteiras, grava a imagem resultante em JPEG com qualidade 90, gera uma miniatura de 128x128 e calcula o SHA-256 do arquivo gerado. O filtro de mediana responde por cerca de dois terços do tempo de cada imagem, e é ele que sustenta a escolha de processos em vez de threads, discutida na seção 2.

Nenhuma imagem depende de outra e as tarefas não trocam dados entre si, o que caracteriza um paralelismo de dados puro. O único resultado combinado é o histograma global do lote: uma soma de inteiros, operação associativa e comutativa, que pode ser feita em qualquer ordem sem alterar o resultado. Média e desvio-padrão de cada canal são calculados uma única vez, ao final, a partir desse histograma. Manter apenas inteiros no estado compartilhado é uma decisão de projeto: a soma de números de ponto flutuante depende da ordem das parcelas, e em paralelo a ordem de chegada muda a cada execução, o que faria a versão paralela divergir da sequencial no último dígito.

3.334 imagens, ou 873.988.096 pixels processados e cerca de 320 MB gravados por execução, entre imagens e miniaturas. A versão sequencial leva $252,57 \pm 25,49$ s (5 execuções) na máquina da demonstração, o que equivale a 75,8 ms por imagem. Desse total, as fases de preparo e de gravação final somam menos de 0,2% do tempo, o que dá uma fração paralelizável $f = 0,99944$, usada no teto de Amdahl da seção 5.

As versões sequencial e paralela chamam a mesma `processar_imagem()`, então produzem os mesmos bytes por construção. Cada execução grava um `manifesto.txt` com o

nome e o SHA-256 de cada imagem gerada, em ordem alfabética, e um `estatisticas.json` com o histograma global, os contadores e uma impressão digital do lote, que é o SHA-256 do manifesto somado ao histograma e aos contadores. O programa `verificar.py` compara duas execuções campo a campo e termina com código 0 quando são idênticas e 1 quando divergem; ele foi testado também no caso de falha, com uma imagem a menos, e acusou corretamente a divergência. Nas 27 execuções da bateria de medições, incluindo as versões com 2, 4 e 8 processos, com threads e com trava grossa, a impressão digital foi sempre a mesma: `ed067b987c6c422c`.

2. Estratégia de paralelização

O programa usa paralelismo de dados no padrão mestre-trabalhadores: o processo principal cria um `multiprocessing.Pool` com `p` processos e distribui as 3.334 imagens da entrada entre eles pelo método `imap_unordered`, com `chunksize=4`. Cada processo executa a mesma função `processar_imagem()` da versão sequencial, e o único ponto de contato entre eles é a soma do histograma no estado global, descrita na seção 3.

Como as imagens não têm relação entre si, a ordem em que os resultados chegam não importa: `imap_unordered` entrega cada resultado assim que fica pronto, então um processo que termina cedo recebe mais trabalho, sem esperar pelos outros. Essa é a distribuição dinâmica que equilibra a carga entre os fluxos. O `chunksize=4` agrupa 4 imagens por envio, reduzindo o número de idas e vindas pelo canal entre processos; pelo canal passam apenas o caminho de cada imagem e, na volta, o nome, o SHA-256 e um identificador do fluxo. A biblioteca usada é `multiprocessing.Pool`, e não `ProcessPoolExecutor`, porque o `Pool` tem um equivalente para threads com a mesma interface (`ThreadPool`), o que permite comparar processos e threads trocando uma única opção do `par.py` (`--modo processos|threads`).

No Windows, o método de início é `spawn`: cada processo filho é um `python.exe` novo que importa os módulos do zero e não herda variáveis do processo pai. A trava e a memória compartilhada (o estado global de `estado.py`) são passadas pelo `initializer` do `Pool`, que roda uma vez em cada processo filho antes de ele receber a primeira tarefa. Passar a trava como argumento de uma tarefa, em vez do `initializer`, dá `RuntimeError: Lock objects should only be shared between processes through inheritance`, porque o `Lock` não pode ser serializado entre processos independentes.

Processos, e não threads: por que, com números. Em CPython, o GIL permite que apenas uma thread execute código Python por vez dentro de um mesmo processo. O `Pillow` libera o GIL durante boa parte das suas operações em C (decodificar, redimensionar, aplicar

nitidez), mas o filtro de mediana 3x3 — cerca de dois terços do tempo de cada imagem — mantém o GIL preso enquanto ordena os 9 valores de vizinhança de cada pixel e cada canal. Como esse filtro é a etapa dominante do pipeline, o trabalho é limitado por CPU e não escala com threads dentro de um único processo.

A bateria de medições confirma isso na prática. Com 8 fluxos na mesma máquina (4 núcleos físicos, 8 lógicos):

Tabela 1 - Processos x threads com 8 fluxos

Configuração	Tempo total	Speedup
Sequencial (referência)	252,57 ± 25,49 s	1,00x
Processos, p = 8	76,07 ± 2,89 s	3,32x
Threads, p = 8	189,94 ± 1,23 s	1,33x

Fonte: Autores (2026).

Processos deram 3,32x de speedup; threads, apenas 1,33x, porque as threads continuam disputando o mesmo interpretador e o mesmo GIL para a etapa que mais pesa no tempo total. Cada processo, em vez disso, tem o seu próprio interpretador e o seu próprio GIL, e por isso o ganho de processos se aproxima do número de núcleos físicos disponíveis, enquanto o de threads fica limitado à fração do tempo que o Pillow consegue liberar o GIL. Essa é a justificativa, pela natureza do trabalho, para escolher processos.

A trava protege apenas a soma do histograma no estado global (seção 3), não o processamento da imagem inteira. O experimento com a trava em volta de todo o processamento (--trava grossa) mostra o efeito de errar essa granularidade: com 8 processos, o tempo total sobe para 239,75 ± 4,28 s, um speedup de apenas 1,05x, quase igual à versão sequencial, porque os processos passam a maior parte do tempo esperando a trava em vez de trabalhar em paralelo.

3. Seção crítica e primitiva

Todos os processos escrevem no mesmo histograma RGB global (768 inteiros de 64 bits) e em 4 contadores (imagens processadas, pixels processados, nanossegundos esperando a trava e nanossegundos com a trava na mão). Essa memória é criada em `ctx.RawArray("q", ...)`, que é compartilhada entre processos mas não tem trava própria: qualquer processo pode ler e escrever nela livremente, e é por isso que a soma precisa de proteção explícita.

A operação `_hist[:] += hist_local` não é atômica: na prática, é ler o valor atual do vetor, somar o histograma local e escrever o resultado de volta. Se dois processos fazem essa

sequência ao mesmo tempo, os dois leem o mesmo valor antigo, e o processo que escreve por último apaga a soma do outro. É a atualização perdida, e ela existe porque o += sobre memória compartilhada, sem trava, não garante exclusão mútua entre os processos.

A primitiva usada é um multiprocessing.Lock, criado uma vez no processo principal e propagado aos processos filhos pelo initializer do Pool. A seção crítica está nas linhas 37 a 42 de estado.py, dentro da função mesclar():

Figura 1 - Seção crítica da função mesclar() em estado.py

```
def mesclar(hist_local, n_pixels: int) -> None:
    t0 = time.perf_counter_ns()
    with _trava:
        t1 = time.perf_counter_ns()
        somar_sem_protecao(hist_local, n_pixels)
        _cont[ESPERA_NS] += t1 - t0
        _cont[DENTRO_NS] += time.perf_counter_ns() - t1
```

Fonte: Autores (2026).

Só as três linhas dentro do with _trava: formam a seção crítica: somar o histograma local no global e atualizar os dois contadores. O processamento da imagem inteira — decodificar, filtrar, salvar, calcular o SHA-256 — fica fora da trava e roda em paralelo sem qualquer sincronização, porque cada processo trabalha sobre a sua própria cópia local da imagem até o momento de mesclar.

O estado global é um único recurso, e o requisito é exclusão mútua simples: só um processo por vez pode somar nele. Um Lock é exatamente um semáforo binário (contador 0/1). Um semáforo com contador maior que 1 permitiria que vários processos entrassem na seção crítica ao mesmo tempo, o que reintroduziria a corrida. Um monitor — uma trava associada a uma variável de condição — seria necessário se algum processo precisasse esperar por uma condição além da posse da trava (por exemplo, esperar o estado atingir um certo valor antes de prosseguir), o que não acontece aqui: cada processo só quer entrar, somar e sair.

Existe uma única trava no programa inteiro, sempre adquirida com with _trava: (que garante a liberação mesmo se somar_sem_protecao lançar uma exceção) e nunca adquirida dentro de outra trava. Sem múltiplas travas mantidas simultaneamente por diferentes processos, não existe espera circular, condição necessária para deadlock.

Para expor a corrida de forma confiável — no programa real ela é rara, porque a seção crítica dura microssegundos a cada dezenas de milissegundos de processamento — o teste usa 4 processos fazendo 20.000 somas cada um sobre o mesmo estado, sincronizados por

uma Barrier para começarem juntos e disputarem de verdade. O resultado esperado é sempre contador = 80.000 e cada posição do histograma = 80.000. Rodado nesta máquina:

Figura 2 - Saída do teste_corrida.py sem e com trava

```
4 processos x 20000 somas cada | esperado: contador = 80000 e soma do histograma = 61440000
SEM trava | rodada 1: contador 48111/80000 | posições erradas 768/768 | PERDEU ATUALIZAÇÕES (0.5 s)
SEM trava | rodada 2: contador 57253/80000 | posições erradas 768/768 | PERDEU ATUALIZAÇÕES (0.5 s)
SEM trava | rodada 3: contador 55793/80000 | posições erradas 768/768 | PERDEU ATUALIZAÇÕES (0.5 s)
COM trava | rodada 1: contador 80000/80000 | posições erradas 0/768 | OK (1.4 s)
COM trava | rodada 2: contador 80000/80000 | posições erradas 0/768 | OK (1.5 s)
COM trava | rodada 3: contador 80000/80000 | posições erradas 0/768 | OK (1.5 s)
```

Fonte: Autores (2026).

Sem a trava, o contador final variou a cada rodada (48.111, 57.253, 55.793 de 80.000 esperado), e as 768 posições do histograma erraram em todas as rodadas: a atualização perdida acontece na prática, não é só uma possibilidade teórica. Com a trava, as três rodadas bateram exatamente em 80.000, sem nenhuma posição errada. O teste foi repetido em execuções independentes e o padrão se manteve: toda rodada sem trava perde atualizações e toda rodada com trava acerta.

Na bateria oficial (3.334 imagens reais, não o teste de estresse), a soma no estado global custa muito pouco frente ao processamento: com 8 processos, a espera total pela trava foi de apenas 45,5 ms ao longo de uma execução de 76 s (cerca de 0,06% do tempo), e a ocupação dos fluxos ficou em 99,4%. Essa é a evidência de que a granularidade fina da trava — proteger só a soma, não o processamento — é o que permite o speedup de 3,32x discutido na seção 2: a sincronização em si não é o fator limitante do ganho.

4. Recursos provisionados

Toda a execução, medição e demonstração acontecem numa máquina local preparada pela equipe, sem uso de nuvem (premissa combinada com o professor). O equivalente ao "grupo de segurança" pedido na lauda é o Firewall do Windows, configurado por regras próprias do projeto.

Tabela 2 - Máquina da demonstração

Item	Valor
Processador	11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz
Núcleos físicos	4
Núcleos lógicos (threads de hardware)	8
Memória RAM	24 GB

Sistema operacional	Windows 11
Python	3.13.15 (instalação padrão, com GIL)
Pillow	12.3.0
NumPy	2.5.3

Fonte: Autores (2026).

A ficha completa, gerada automaticamente por `ficha_maquina.py`, fica em `infra/ficha_maquina.md`. A frequência de clock relatada pelo Windows em repouso (1382 MHz) é menor que a base de 2,40 GHz porque o sistema reduz o clock quando a CPU está ociosa; durante o processamento, o Turbo Boost eleva a frequência acima da base, o que é discutido como um dos fatores de hardware na seção 6.

As regras de entrada do Firewall do Windows funcionam como o equivalente local de um grupo de segurança na nuvem: por padrão, toda conexão de entrada é bloqueada, e só as portas explicitamente liberadas aceitam tráfego.

Tabela 3 - Regras de acesso do Firewall do Windows

Porta	Protocolo	O que fica nela	Origem permitida
8000	TCP	Serviço: página de resultados (python -m http.server 8000)	Qualquer origem (é o serviço)
22	TCP	Administração: SSH (OpenSSH Server do Windows)	Só o IP de quem administra
Todas as outras	—	—	Bloqueadas (ação padrão de entrada = Bloquear)

Fonte: Autores (2026).

As regras são criadas por `infra/firewall_windows.ps1 -IpEquipe <ip>`, rodado como administrador, e ficam agrupadas sob o nome "Lote Imagens" no Firewall do Windows com Segurança Avançada (`wf.msc`), o que permite mostrá-las ao vivo na apresentação. O script também restringe a regra `OpenSSH-Server-In-TCP`, criada automaticamente pela instalação do OpenSSH Server, que por padrão não limita a origem.

No dia da apresentação, o IP autorizado na porta 22 é atualizado para o IP da rede da sala de aula (o endereço muda conforme a rede), e a máquina volta a ficar apenas com a porta 8000 aberta para o público e a 22 restrita à equipe. Os detalhes de cada regra, incluindo o IP configurado no momento, ficam em `infra/regras_de_acesso.md`.

A máquina tem 4 núcleos físicos e 8 lógicos, por Hyper-Threading (dois fluxos de hardware compartilhando as unidades de execução de cada núcleo físico). Isso explica por que o speedup medido cresce de forma quase linear até $p = 4$ e depois desacelera até $p = 8$, discutido nas seções 5 e 6: acima do número de núcleos físicos, os fluxos adicionais competem pelas mesmas unidades de execução, em vez de ganhar capacidade de processamento nova.

5. Medição e speedup

Todas as medições foram feitas na máquina da seção 4, com a mesma entrada (3.334 imagens) e o mesmo código. A máquina ficou na tomada, em modo de alto desempenho e sem outros programas abertos. O programa `bench.py` executou uma rodada de aquecimento, descartada, e depois 5 rodadas, cada uma com a versão sequencial seguida das paralelas com $p = 2, 4, 8$ processos. Em seguida rodaram os experimentos com 8 threads e com trava grossa em 8 processos, 3 rodadas cada. Cada execução rodou como um processo separado, com a pasta de saída limpa antes, fora da medição. Todas as 27 execuções produziram a mesma impressão digital (ed067b987c6c422c).

Tabela 4 - Tempos, speedup e eficiência.

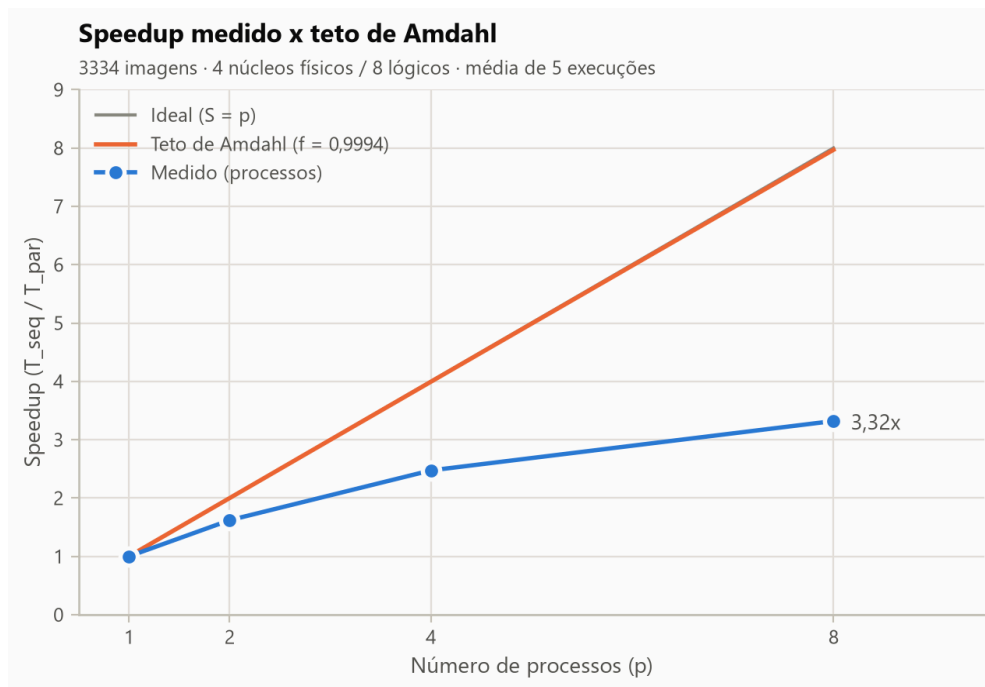
Configuração	Tempo total (s)	Speedup	Eficiência	Teto de Amdahl	Karp–Flatt
Sequencial	252,57 ± 25,49 (n=5)	1,00	100%	—	—
Processos, $p = 2$	155,93 ± 10,61 (n=5)	1,62	81,0%	2,00	0,2347
Processos, $p = 4$	102,10 ± 10,68 (n=5)	2,47	61,8%	3,99	0,2056
Processos, $p = 8$	76,07 ± 2,89 (n=5)	3,32	41,5%	7,97	0,2014
Threads, $p = 8$	189,94 ± 1,23 (n=3)	1,33	16,6%	7,97	0,7166
Trava grossa, $p = 8$	239,75 ± 4,28 (n=3)	1,05	13,2%	7,97	0,9420

Fonte: Autores (2026).

A versão sequencial mede separadamente o preparo, o processamento e a etapa final. A fração paralelizável estimada é $f = T_{\text{processamento}} / T_{\text{total}} = 0,99944$ (média de 5 execuções): as fases de preparo e gravação final juntas somam menos de 0,2% do tempo

sequencial. Com 8 processos, o teto de Amdahl é $S_{\text{max}} = 1 / ((1 - f) + f/8) = 7,97$ e o speedup medido foi $S = 3,32$: 41,7% do teto.

Figura 3 - Speedup medido x teto de Amdahl x ideal



Fonte: Autores (2026).

6. O que limitou o ganho

Como a fração paralelizável é muito próxima de 1 ($f = 0,99944$), o teto de Amdahl fica perto de p , e a diferença para o speedup medido vem de sobrecarga e de hardware, e não de uma parte serial fixa do programa. A métrica de Karp–Flatt mostra isso de outro ângulo: $e(p) = 0,2347, 0,2056$ e $0,2014$ para $p = 2, 4$ e 8 — praticamente constante, e até levemente decrescente com p , em vez de crescente. Isso indica que a sobrecarga não piora proporcionalmente quando mais processos são acrescentados; ela é sobretudo um custo fixo (criar os processos, abrir o pool), diluído por um número maior de fluxos.

Tabela 5 - Fontes de sobrecarga por configuração

Configuração	Espera pela trava (ms)	Início da 1ª tarefa (s)	Cauda (s)	Ocupação
Processos, $p = 2$	22,3	0,42	0,15	99,9%
Processos, $p = 4$	27,0	0,50	0,36	99,7%
Processos, $p = 8$	45,5	0,64	0,52	99,4%
Threads, $p = 8$	14,2	0,03	0,68	99,8%

Trava grossa, p = 8	1.666.565,0	0,68	0,82	99,7%
---------------------	-------------	------	------	-------

Fonte: Autores (2026).

- **Comunicação e criação dos processos:** com 8 processos, a primeira tarefa só começa 0,64 s depois do início do processamento, tempo gasto criando os processos (no Windows, cada um é um novo interpretador que importa Pillow e NumPy do zero, por causa do spawn). Pelo canal entre processos passam apenas o caminho de cada imagem e, na volta, o nome e o SHA-256: um volume de dados pequeno perto do tempo de processamento de cada imagem.
- **Divisão desigual do trabalho:** no fim do lote, os processos terminam em momentos diferentes; a cauda foi de 0,52 s com 8 processos. O chunksize de 4 e o balanceamento dinâmico do `imap_unordered` mantêm essa cauda pequena frente ao tempo total (76 s), mas ela ainda pesa mais, proporcionalmente, à medida que o tempo total cai com mais processos.
- **Espera na seção crítica:** com a trava fina, apenas 45,5 ms de espera acumulada ao longo de uma execução de 76 s (cerca de 0,06% do tempo), desprezível. Com a trava em volta do processamento inteiro (trava grossa), a espera acumulada sobe para 1.666,6 s — os processos passam a maior parte do tempo enfileirados esperando a trava, em vez de trabalhar em paralelo — e o speedup despenca para 1,05x, quase o mesmo da versão sequencial.
- **Hardware:** a máquina tem 4 núcleos físicos e 8 lógicos (Hyper-Threading). Até $p = 4$ o ganho é quase linear (speedup 2,47x com eficiência de 61,8%), mas de $p = 4$ para $p = 8$ o ganho desacelera (speedup sobe só para 3,32x, eficiência cai para 41,5%), porque os 4 fluxos extras passam a dividir as mesmas unidades de execução dos núcleos físicos já ocupados, em vez de ganhar capacidade de processamento nova. O Turbo Boost também favorece a execução sequencial: com um só núcleo ativo, a CPU roda numa frequência mais alta do que quando todos os núcleos estão ocupados, o que reduz um pouco a vantagem relativa da paralela.

A versão paralela alcançou 3,32x de speedup com 8 processos, 41,7% do teto de Amdahl para essa fração paralelizável, e o principal limitador não foi a sincronização (a espera pela trava é desprezível na configuração correta), e sim o Hyper-Threading acima dos 4 núcleos físicos, combinado com o custo fixo de criar os processos no Windows. Os experimentos de controle confirmam a escolha de projeto: threads entregam só 1,33x, porque o filtro de mediana segura o GIL, e uma trava mal dimensionada (em volta do processamento

inteiro) reduz o ganho a 1,05x, mostrando que tanto a escolha de processos quanto a granularidade fina da trava são necessárias para o speedup observado.